Лабораторная работа №5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

1. Параметры экспериментов и характеристики компьютера

Измерения проводились на Apple M2 (8 ядер/8 потоков). Компилятор gcc-16. Флаги: `-O3 -march=native -fopenmp`. Время — `omp_get_wtime()`. Базовые параметры: $N=10^5$, d=32, K=50, $\lambda=0.1$, $\eta=0.01$.

3.1. Проверка корректности

Реализация	Итоговая функция потерь	\(\Delta w\)	Корректность подтверждена?
Последовательная	2.0482	0.00e+00	Да (эталон)
OpenMP	2.0482	1.94e-16	Да
SIMD	2.0482	0.00e+00	Да
OpenMP + SIMD	2.0482	1.67e-16	Да

3.2. Сравнение реализаций ($N=10^5$, d=32, K=50)

Вариант	Время, с	Ускорение S
Последовательная версия	0.25393	1.00
OpenMP	0.10494	2.42
SIMD	0.14163	1.79
OpenMP + SIMD	0.10238	2.48

3.3. Влияние числа объектов N (d=32, K=50)

N	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP + SIMD, с
10^4	0.02001	0.01325	0.01285	0.01047
10^5	0.19475	0.11230	0.12937	0.10266
10^6	1.81885	0.88746	1.17889	0.93716

3.4. Влияние числа признаков d ($N=10^5$, K=50)

d	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP + SIMD, с
8	0.06486	0.02257	0.05833	0.02075
32	0.18173	0.08450	0.11290	0.05817
128	0.99724	0.26583	0.53847	0.17069
256	2.36331	1.02845	1.27914	0.74227

3.5. Влияние числа потоков ($N=10^5$, d=32)

База для S_p — однопоточные версии. $S_p = T_1 / T_p$, $E_p = S_p / p$.

Потоки p	OpenMP			OpenMP + SIMD		
	Время, с	S_p	E_p	Время, с	S_p	E_p
1	0.17226	1.00	1.00	0.11240	1.00	1.00
2	0.09465	1.82	0.91	0.06379	1.76	0.88
4	0.05755	2.99	0.75	0.08154	1.38	0.34
8	0.10658	1.62	0.20	0.06691	1.68	0.21

3.6. Влияние коэффициента регуляризации λ ($N=10^5$, d=32, OpenMP+SIMD)

λ	Итоговая функция потерь F(w,b)	Число коэффициентов $\lvert w_j\rvert < \varepsilon$	Время обучения, с
0	1.9612	0	0.05975
10^{-4}	1.9617	0	0.09905
10^{-3}	1.9658	2	0.05174
10^{-2}	2.0062	16	0.06141
10^{-1}	2.3491	16	0.08809

3.7. Влияние флагов компиляции ($N=10^5$, d=32)

Флаги компиляции	Время, с	Ускорение	Примечания
`-O0 -fopenmp`	0.25018	1.00	Без оптимизаций и векторизации
`-O3 -fopenmp`	0.06500	3.85	Автоматическая векторизация
`-O3 -march=native -fopenmp`	0.05714	4.38	Использование AVX/NEON
`-O3 -march=native -fopenmp + omp simd`	0.05714	4.38	Принудительная векторизация (в коде)

3.8. Анализ результатов

1. Самая быстрая реализация. На малых и средних размерах ($N \le 10^5$, $d \le 128$) быстрее всего `OpenMP + SIMD` (S = 2.48). При $N = 10^6$ чистый `OpenMP` обгоняет его (0.887 с против 0.937 с) из-за накладных расходов на векторизацию при больших объёмах данных.

2. OpenMP vs SIMD. Ускорение OpenMP (2.42) значительно выше, чем у чистого SIMD (1.79), так как многопоточность задействует все ядра, а SIMD — только векторные блоки одного ядра. Ускорение меньше p из-за закона Амдала (последовательные обновления весов) и насыщения шины памяти.

3. Эффективность от потоков. При p=2 $E_p = 0.91$, но при p=4 падает до 0.75, а при p=8 — до 0.20. Более того, при p=8 время хуже, чем при p=4 (0.106 с против 0.057 с). Причина — **насыщение пропускной способности памяти** и рост накладных расходов на синхронизацию (false sharing при записи в массив e).

4. Векторизация. Автоматически (и с `omp simd`) векторизуется внутренний цикл по \(j\) при вычислении предсказаний и градиентов. Цикл накопления `g[j] += e[i] * X[i*d+j]` без `omp simd` не векторизуется из-за возможных зависимостей (alias). `omp simd` снимает эти ограничения и разрешает редукцию.

5. Влияние λ. С ростом λ функция потерь растёт (от 1.9612 до 2.3491) из-за штрафа. Число нулевых коэффициентов увеличивается: при $\lambda=10^{-3}$ обнуляются 2, при $\lambda=10^{-2}$ и выше — уже 16 (половина признаков). Это подтверждает свойство L1-регуляризации отсекать неинформативные признаки.

6. Ограничения производительности. Основной ограничитель — доступ к памяти. При $N=10^6$ матрица X занимает 256 МБ, что упирается в пропускную способность RAM. Также сказываются накладные расходы OpenMP на слияние локальных градиентов и последовательные обновления весов. Флаги `-O3 -march=native` дают ускорение в 4.38 раза по сравнению с `-O0`.